

Chapter 15: Greedy Algorithms

Introduction to Algorithms (CLRS, 4th Edition)

Asaduzzaman Herok

Department of Computer Science & Engineering

Islamic University of Technology

June 2, 2026

This slide is generated with the help of Claude.ai

■ Section 1

Introduction to Greedy Algorithms

What is a Greedy Algorithm?

Core Idea

A greedy algorithm always makes the choice that looks **best at the moment** — a *locally optimal* choice — in the hope of finding a *globally optimal* solution.

- Algorithms for **optimization problems** go through a sequence of steps, with choices at each step.
- Dynamic programming considers *all* choices; greedy picks *one* — the locally best one.
- Greedy is simpler & more efficient than DP when applicable.

Important Caveat

Greedy algorithms do **not** always yield optimal solutions. We must **prove** that greedy works for each specific problem.

Analogy

Imagine navigating a maze: a greedy navigator always turns toward the direction that *looks closest* to the exit — sometimes it works perfectly, sometimes it leads to dead ends.

Why Study Greedy Algorithms?

Topics Covered in This Chapter

- ➊ **Activity-Selection Problem** — scheduling problems (§15.1)
- ➋ **Elements of the Greedy Strategy** — when & why greedy works (§15.2)
- ➌ **Huffman Codes** — data compression (§15.3)
- ➍ **Offline Caching** — furthest-in-future strategy (§15.4)

Broader Applications (Later Chapters may get covered)

- **Minimum Spanning Trees** (Ch. 21) — Prim's & Kruskal's algorithms
- **Single-Source Shortest Paths** (§22.3) — Dijkstra's algorithm
- **Set-Cover Heuristic** (§35.3) — approximation algorithms

Greedy vs Dynamic Programming — Quick Comparison

Feature	Dynamic Programming	Greedy
Choice	Depends on subproblem solutions	Locally optimal at each step
Order	Bottom-up (or top-down+memo)	Top-down
Subproblems	Many overlapping subproblems	Single remaining subproblem
Correctness	Always optimal (with DP)	Must prove greedy-choice property
Efficiency	Generally slower	Generally faster
Requirement	Optimal substructure	Optimal substructure + greedy-choice property

■ Section 2

The Activity-Selection Problem

Problem Statement

Setting

- A **set** $S = \{a_1, a_2, \dots, a_n\}$ of n activities compete for a **single shared resource** (e.g., a conference room).
- Each activity a_i has a **start time** s_i and a **finish time** f_i , where $0 \leq s_i < f_i < \infty$.
- Activity a_i occupies the interval $[s_i, f_i)$.

Compatibility

Activities a_i and a_j are **compatible** if their intervals do not overlap:

$$s_i \geq f_j \quad \text{or} \quad s_j \geq f_i$$

Goal

Select a **maximum-size subset** of mutually compatible activities.

Example: Activity Table I

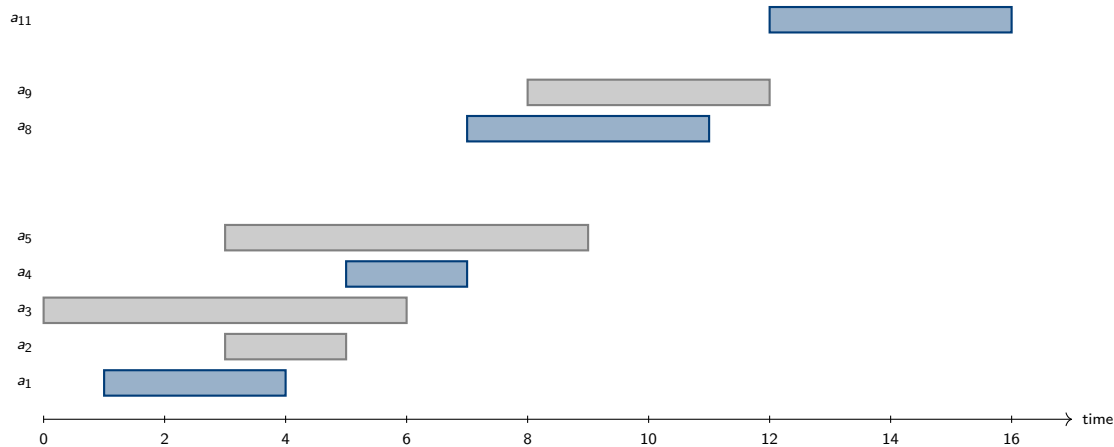
i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	7	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

Table: Activities sorted by finish time.

Compatible Sets

- $\{a_3, a_9, a_{11}\}$ — compatible, *but not maximum*
- $\{a_1, a_4, a_8, a_{11}\}$ — **maximum-size** (4 activities)
- $\{a_2, a_4, a_9, a_{11}\}$ — also maximum-size

Example: Activity Table II



Blue = optimal set $\{a_1, a_4, a_8, a_{11}\}$

Optimal Substructure I

Definition

Let S_{ij} = activities that *start after* a_i finishes and *finish before* a_j starts.

$$S_{ij} = \{a_k \in S \mid f_i \leq s_k \text{ and } f_k \leq s_j\}$$

Recurrence (Dynamic Programming)

Let $c[i, j]$ = size of a maximum compatible subset of S_{ij} :

$$c[i, j] = \begin{cases} 0 & \text{if } S_{ij} = \emptyset \\ \max_{a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & \text{if } S_{ij} \neq \emptyset \end{cases}$$

Cut-and-Paste Argument

If A_{ij} is an optimal solution and includes activity a_k , then:

- $A_{ik} = A_{ij} \cap S_{ik}$ must be optimal for S_{ik}
- $A_{kj} = A_{ij} \cap S_{kj}$ must be optimal for S_{kj}

Otherwise we could substitute a better sub-solution — contradiction!

But do we need to solve all subproblems? Can we be smarter?

Making the Greedy Choice I

Key Observation

Instead of solving all subproblems, we make **one greedy choice**:
Always pick the activity with the **earliest finish time**

Why earliest finish time?

- Leaves the resource available for as **many future activities** as possible.
- After choosing a_1 , the only remaining subproblem is:

$$S_1 = \{a_i \in S \mid s_i \geq f_1\}$$

- No need to consider activities finishing before a_1 starts — since $s_1 < f_1$, no activity can finish before f_1 .

Greedy Property

Activities sorted by finish time $\Rightarrow$ greedy choice is **always** a_1 (or first in current subproblem).

Contrast with DP

DP examines all $a_k \in S_{ij}$ to decide the best split. Greedy skips this — just pick the earliest finish!

Theorem 15.1 — Greedy Choice is Safe

Theorem 15.1

Consider any nonempty subproblem S_k , and let a_m be the activity in S_k with the **earliest finish time**. Then a_m is included in **some maximum-size** subset of mutually compatible activities of S_k .

Proof (Exchange Argument)

Let A_k be a maximum-size compatible subset of S_k .

Let $a_j =$ activity in A_k with earliest finish time.

- **Case 1:** $a_j = a_m \Rightarrow$ Done! $a_m \in A_k$.
- **Case 2:** $a_j \neq a_m \Rightarrow$ Construct $A'_k = (A_k \setminus \{a_j\}) \cup \{a_m\}$.
 - Since $f_m \leq f_j$ and activities in A_k are compatible, A'_k is also compatible.
 - $|A'_k| = |A_k| \Rightarrow A'_k$ is also maximum-size.
 - A'_k includes a_m .



RECURSIVE-ACTIVITY-SELECTOR

```
1: function RECURSIVE-ACTIVITY-SELECTOR( $s, f, k, n$ )
2:    $m \leftarrow k + 1$ 
3:   while  $m \leq n$  and  $s[m] < f[k]$  do                                ▷ Find first activity in  $S_k$  to finish
4:      $m \leftarrow m + 1$ 
5:   if  $m \leq n$  then
6:     return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ 
7:   else
8:     return  $\emptyset$ 
```

Recursive Greedy Algorithm II

- Add fictitious activity a_0 with $f_0 = 0$.
- Initial call: $\text{RAS}(s, f, 0, n)$
- **Running time:** $\Theta(n)$ — each activity is examined *exactly once* across all recursive calls.

Execution Trace (Example)

$\text{RAS}(s, f, 0, 11) \rightarrow$ selects a_1

$\text{RAS}(s, f, 1, 11) \rightarrow$ selects a_4

$\text{RAS}(s, f, 4, 11) \rightarrow$ selects a_8

$\text{RAS}(s, f, 8, 11) \rightarrow$ selects a_{11}

$\text{RAS}(s, f, 11, 11) \rightarrow$ returns $\emptyset$

Result: $\{a_1, a_4, a_8, a_{11}\}$

GREEDY-ACTIVITY-SELECTOR

```
1: function GREEDY-ACTIVITY-SELECTOR( $s, f, n$ )
2:    $A \leftarrow \{a_1\}$ 
3:    $k \leftarrow 1$ 
4:   for  $m \leftarrow 2$  to  $n$  do
5:     if  $s[m] \geq f[k]$  then
6:        $A \leftarrow A \cup \{a_m\}$ 
7:        $k \leftarrow m$ 
8:   return  $A$ 
```

▷ Always select first activity

▷ k = index of most recently added activity

▷ Is a_m compatible with last chosen?

Iterative Greedy Algorithm II

Key Invariant

$f_k = \max\{f_i : a_i \in A\}$ — the finish time of a_k is always the *latest* in A . So compatibility check reduces to a **single comparison**: $s[m] \geq f[k]$.

Complexity

- **Pre-sort:** $\mathcal{O}(n \log n)$
- **Main loop:** $\Theta(n)$
- **Total:** $\mathcal{O}(n \log n)$
- Identical result to recursive version

Step-by-Step Execution on Example

Step	m	$s[m]$	$f[k]$	Compatible?	Action
Init	—	—	0	—	$A = \{a_1\}, k = 1, f[k] = 4$
1	2	3	4	No ($3 < 4$)	Skip a_2
2	3	0	4	No ($0 < 4$)	Skip a_3
3	4	5	4	Yes ($5 \geq 4$)	Add $a_4, k = 4, f[k] = 7$
4	5	3	7	No ($3 < 7$)	Skip a_5
5	6	5	7	No ($5 < 7$)	Skip a_6
6	7	6	7	No ($6 < 7$)	Skip a_7
7	8	7	7	Yes ($7 \geq 7$)	Add $a_8, k = 8, f[k] = 11$
8	9	8	11	No ($8 < 11$)	Skip a_9
9	10	2	11	No ($2 < 11$)	Skip a_{10}
10	11	12	11	Yes ($12 \geq 11$)	Add $a_{11}, k = 11$

Result: $A = \{a_1, a_4, a_8, a_{11}\}$ (**Optimal!**)

Activity-Selection: Summary & Key Takeaways

Design Steps Used

- 1 Identify **optimal substructure** → derived DP recurrence $c[i, j]$
- 2 Observe that **one greedy choice** (earliest finish) reduces to one subproblem
- 3 Proved greedy is safe via **exchange argument** (Theorem 15.1)
- 4 Converted recursive greedy to **iterative** algorithm

Greedy Insight

Among all activities, always picking the one that finishes first *maximizes room* for future selections — this is the **intuition** behind the greedy choice.

Other Greedy Choices — Do They Work?

- Earliest start time? **No!**
- Shortest duration? **No!**
- Fewest conflicts? **No!**
- Latest start time? **Yes!** (Exercise 15.1-2)

■ Section 3

Elements of the Greedy Strategy

Two Key Ingredients for Greedy Algorithms

➤ 1. Greedy-Choice Property

A **globally optimal** solution can be assembled by making *locally optimal* (greedy) choices.

The choice is made **without** considering solutions to subproblems — it depends only on the current situation.

Key: Greedy chooses first, then solves the subproblem.

➤ 2. Optimal Substructure

An optimal solution to the problem **contains within it** optimal solutions to subproblems.

Same property required by dynamic programming.

Key: After the greedy choice, the remaining subproblem combined with the choice yields the global optimum.

Proof Strategy for Correctness

Show both properties hold $\implies$ greedy algorithm is correct.

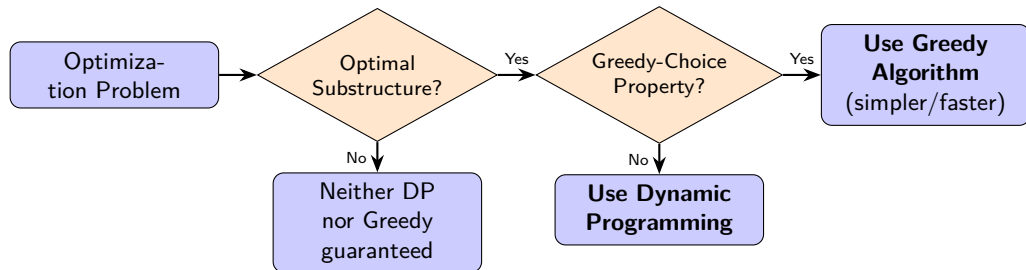
Standard Steps to Design a Greedy Algorithm

- 1 **Cast** the optimization problem as one where you make a **single choice** and are left with **one subproblem** to solve.
- 2 **Prove** there is always an optimal solution that includes the greedy choice (*greedy-choice property*). Use an **exchange argument**: take any optimal solution, swap the greedy choice in, show cost doesn't increase.
- 3 **Demonstrate optimal substructure**: after making the greedy choice, the remaining subproblem combined with the choice yields an optimal solution to the original problem.

Underlying Reality

Beneath every greedy algorithm, there is almost always a more cumbersome DP solution. Greedy is an optimized special case.

Greedy vs Dynamic Programming: Decision Flow



Case Study: 0-1 Knapsack vs Fractional Knapsack I

Problem Setup

A thief has a knapsack of capacity W pounds. n items, item i : value v_i , weight w_i .

- **0-1 Knapsack:** Must take or leave entire item (binary choice).
- **Fractional Knapsack:** Can take any fraction of an item.

Fractional Knapsack — Greedy Works!

Greedy strategy:

- 1 Compute value-per-pound v_i/w_i for each item.
- 2 Take as much as possible of highest v_i/w_i item.
- 3 Continue until knapsack is full.

Time: $\mathcal{O}(n \log n)$ (sorting)

Greedy-choice property holds!

Case Study: 0-1 Knapsack vs Fractional Knapsack II

0-1 Knapsack — Greedy Fails!

Counterexample ($W = 50$):

- Item 1: 10 lb, \$60 (\$6/lb) ← greedy picks
- Item 2: 20 lb, \$100 (\$5/lb)
- Item 3: 30 lb, \$120 (\$4/lb)

Greedy: $\$60 + \$100 = \$160$ (40 lb)

Optimal: $\$100 + \$120 = \$220$ (50 lb)

Greedy is suboptimal! $\Rightarrow$ Use DP.

Why Does Greedy Fail for 0-1 Knapsack?

The Root Cause

In the 0-1 problem, including an item may leave **unused capacity**. The empty space lowers the effective value-per-pound of the load.

To decide whether to include item j , you must compare:

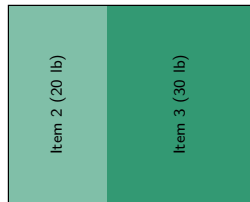
- Subproblem that **includes** item j
- Subproblem that **excludes** item j

This creates **overlapping subproblems** — a hallmark of DP, not greedy.

Knapsack (50 lb)



Greedy: **\$160** (10+20=30lb, 20lb wasted)



Optimal: **\$220** (20+30=50lb, no waste)

■ Section 4

Huffman Codes — Data Compression

Motivation: The Data Compression Problem

Setting

- A data file of **100,000 characters** using only 6 distinct characters: a, b, c, d, e, f.
- Goal: represent the file compactly using a **binary character code**.

	a	b	c	d	e	f	Total bits
Frequency (thousands)	45	13	12	16	9	5	—
Fixed-length (3 bits)	000	001	010	011	100	101	300,000
Variable-length	0	101	100	111	1101	1100	224,000

Variable-length savings

$(45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) \times 1000 = 224,000$ bits. Savings $\approx$ **25%**. This is the **optimal** encoding for this file!

Prefix-Free Codes

Definition

A **prefix-free code** is one in which no codeword is a prefix of any other codeword. Also called *prefix codes*.

Why prefix-free?

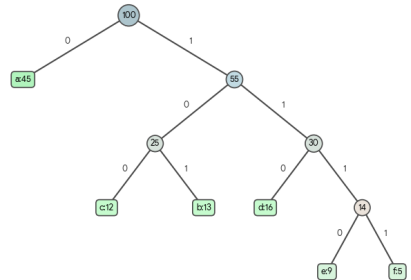
- Simplifies **decoding**: the codeword at the start of an encoded file is unambiguous.
- Identify initial codeword $\rightarrow$ translate $\rightarrow$ repeat on remainder.
- **Encoding** is simple: concatenate codewords.

Example

face encodes as:

$1100 \cdot 0 \cdot 100 \cdot 1101 = 110001001101$

Decoding: $100 \cdot 0 \cdot 1100 \cdot 1101 = \text{cafe}$



Representing Prefix-Free Codes as Binary Trees I

Tree Representation

- **Leaves** = characters of alphabet C
- **Path from root to leaf** = codeword (0=left, 1=right)
- Not a BST — leaves need not be in sorted order
- An optimal code $\Rightarrow$ tree is a **full binary tree** (every internal node has exactly 2 children)
- $|C|$ leaves, $|C| - 1$ internal nodes

Cost of Tree T

$$B(T) = \sum_{c \in C} c.\text{freq} \cdot d_T(c)$$

where $d_T(c)$ = depth of leaf c = length of codeword for c .

Goal: Minimize $B(T)$.

Representing Prefix-Free Codes as Binary Trees II

Fixed-Length vs Optimal

Fixed-length tree: Not full — wastes bits by having codewords starting with 10 but none with 11.

Optimal variable-length tree: Full binary tree — every path corresponds to a valid codeword.

Key insight: *more frequent characters should have shorter paths* (smaller depth = shorter codeword).

Huffman's Algorithm

HUFFMAN(C)

```
1:  $n \leftarrow |C|$ 
2:  $Q \leftarrow C$       ▷ Build min-priority
   queue
3: for  $i \leftarrow 1$  to  $n - 1$  do
4:   allocate new node  $z$ 
5:    $x \leftarrow \text{EXTRACT-MIN}(Q)$ 
6:    $y \leftarrow \text{EXTRACT-MIN}(Q)$ 
7:    $z.\text{left} \leftarrow x$ 
8:    $z.\text{right} \leftarrow y$ 
9:    $z.\text{freq} \leftarrow x.\text{freq} + y.\text{freq}$ 
10:   $\text{INSERT}(Q, z)$ 
11: return  $\text{EXTRACT-MIN}(Q)$ 
```

Key Ideas

- Uses a **min-priority queue** (min-heap) keyed on frequency.
- At each step, **merge** the two least-frequent nodes.
- Merged node's frequency = sum of the two children.
- Performs $n - 1$ merges for n characters.

Running Time

- Build heap: $\mathcal{O}(n)$
- $n - 1$ iterations, each with 2 EXTRACT-MIN + 1 INSERT: $\mathcal{O}(\log n)$ each
- **Total:** $\mathcal{O}(n \log n)$

Huffman Algorithm: Step-by-Step Execution

Initial queue (sorted): f:5, e:9, c:12, b:13, d:16, a:45

Step 1: Merge f:5 + e:9 → z:14

Queue: c:12, b:13, z:14, d:16, a:45

Step 2: Merge c:12 + b:13 → z:25

Queue: z:14, d:16, z:25, a:45

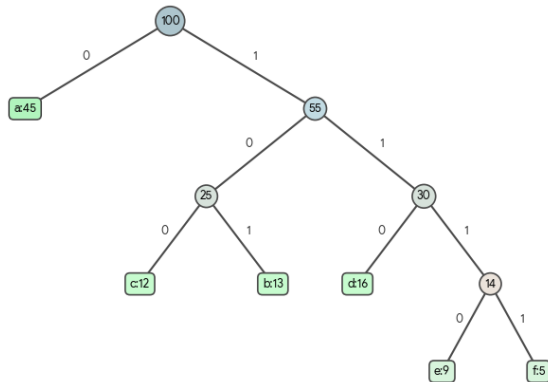
Step 3: Merge z:14 + d:16 → z:30

Queue: z:25, z:30, a:45

Step 4: Merge z:25 + z:30 → z:55

Queue: a:45, z:55

Step 5: Merge a:45 + z:55 → root:100



Resulting codes: a=0, c=100, b=101, d=110, e=1110, f=1111

Correctness of Huffman's Algorithm

Lemma 15.2 — Greedy-Choice Property

Let x and y be the two characters with **lowest frequencies** in C . Then there exists an optimal prefix-free code where x and y have the **same codeword length** and **differ only in the last bit** (i.e., they are siblings at maximum depth).

Proof Sketch (Exchange Argument)

Take any optimal tree T . Let a, b be deepest sibling leaves with $a.\text{freq} \leq b.\text{freq}$ and $x.\text{freq} \leq y.\text{freq}$.

- 1 Swap a and $x \rightarrow$ tree T' : $B(T) - B(T') = (a.\text{freq} - x.\text{freq})(d_T(a) - d_T(x)) \geq 0$
- 2 Swap b and $y \rightarrow$ tree T'' : similarly $B(T') - B(T'') \geq 0$
- 3 Therefore $B(T'') \leq B(T)$, and since T is optimal, $B(T'') = B(T)$.
- 4 T'' is optimal and x, y are sibling leaves of maximum depth. □

Correctness of Huffman's Algorithm (continued)

Lemma 15.3 — Optimal Substructure Property

Let x, y be the two minimum-frequency characters in C . Let $C' = (C \setminus \{x, y\}) \cup \{z\}$ where $z.\text{freq} = x.\text{freq} + y.\text{freq}$.

If T' is an optimal prefix-free code tree for C' , then the tree T obtained by replacing leaf z with an internal node having x and y as children is an optimal prefix-free code for C .

Key Relationship

$$B(T) = B(T') + x.\text{freq} + y.\text{freq} \iff B(T') = B(T) - x.\text{freq} - y.\text{freq}$$

The cost of the merged tree exceeds the cost of the sub-tree by exactly the combined frequency of x and y (the cost of one merge step).

Theorem 15.4

Procedure `HUFFMAN` produces an **optimal prefix-free code**. (Proof: Lemmas 15.2 + 15.3) •



Section 6

Summary & Key Takeaways

Chapter 15: Big Picture Summary

Core Principles

- 1 **Greedy algorithms** make locally optimal choices hoping for a global optimum.
- 2 Correctness requires proving **two properties**:
 - **Greedy-choice property**: locally optimal $\Rightarrow$ globally optimal
 - **Optimal substructure**: optimal solution contains optimal sub-solutions
- 3 Use the **exchange argument**: take any optimal solution, swap in the greedy choice, show it's no worse.

Problem	Greedy Choice	Time	Optimal?
Activity Selection	Earliest finish time	$\Theta(n)$	Yes
Fractional Knapsack	Highest value/weight	$\mathcal{O}(n \log n)$	Yes
0-1 Knapsack	Highest value/weight	—	No!
Huffman Coding	Merge two lowest freq	$\mathcal{O}(n \log n)$	Yes

Designing Greedy Algorithms: Checklist

- ✓ **Identify the greedy choice:** What is the locally best option? (Earliest finish, minimum frequency, furthest future, etc.)
- ✓ **Prove greedy-choice property:** Use exchange argument — swap in greedy choice to any optimal solution; show cost doesn't increase.
- ✓ **Prove optimal substructure:** After making greedy choice, the remaining subproblem + greedy choice = global optimum.
- ✓ **Develop the algorithm:** Usually top-down, often iterative, often uses sorting or a priority queue.
- ✓ **Analyze running time:** Often $\mathcal{O}(n \log n)$ due to sorting or heap operations.
- ✗ **Check edge cases:** Test greedy on examples where it might seem to fail — and prove it doesn't!

⑩ Questions?

Thank you for your attention!

Chapter 15 — Greedy Algorithms

Introduction to Algorithms, 4th Ed. (CLRS)

“A greedy algorithm always makes the choice that looks best at the moment.”

— Cormen, Leiserson, Rivest, Stein